IOT BASED PATIENT MONITORING SYSTEM

SUBMITTED BY

SREYA S	**ATP24AIM023**
VITHUL M	**ATP24AIM024**
PRABHAVATHY J	**IGW24AIM042**

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

PCAML507

INTERNET OF THINGS LAB

Under the Guidance of

Dr. MURUGANANTH GOPAL RAJ

Dean of Research, & Professor

Ahalia School of Engineering & Technology, Palakkad

Academic Year 2026-27 - October 2026

AHALIA SCHOOL OF ENGINEERING & TECHNOLOGY

Approved by AICTE & Affiliated to A. P. J. Abdul Kalam Technological University

ISO 9001: 2015 Certified Institution | NBA accredited B.Tech. CSE & ECE programs (till 31-12-2028)

Ahalia Campus, Palakkad | Ph: 04923 226666 | www.ahalia.ac.in

1. ABSTRACT

The Internet of Things (IoT) has enabled the development of connected healthcare systems capable of collecting & monitoring patient health parameters. This project presents an IoT Based Patient Monitoring System designed to monitor important physiological parameters using an ESP32 microcontroller. The proposed system integrates an AD8232 ECG sensor, pulse sensor & DS18B20 temperature sensor to acquire ECG signals, heart-rate information & body-temperature readings. The ESP32 processes the sensor data & provides a local web-based interface for monitoring the measured parameters. The system also incorporates Wi-Fi connectivity to transmit health data to the ThingSpeak cloud platform, allowing the stored measurements to be accessed & visualized remotely. An LED-based alert mechanism is included for indicating abnormal temperature conditions according to the programmed threshold. The project demonstrates the integration of biomedical sensors, embedded processing, wireless communication & cloud-based data visualization in a single monitoring system. The developed prototype is intended primarily for academic & educational demonstration & does not replace certified medical equipment or professional medical diagnosis.

KEYWORDS: Internet of Things, ESP32, ECG, Heart Rate, Body Temperature, ThingSpeak

2. INTRODUCTION

The Internet of Things (IoT) enables physical devices to collect, process & exchange data through network connectivity. In healthcare, IoT-based systems can integrate sensors, microcontrollers & web interfaces to facilitate the observation of physiological parameters & remote visualization of selected readings. Such systems provide an accessible approach to demonstrating sensor acquisition, embedded processing & wireless communication.

The proposed project, Iot Based Health Monitoring System, uses an ESP32 microcontroller to acquire physiological signals through an AD8232 electrocardiogram (ECG) sensor, a pulse sensor & a DS18B20 temperature sensor. The ESP32 hosts a local web dashboard that displays available sensor readings & visualizes the ECG waveform using Chart.js. Through built-in Wi-Fi connectivity, selected readings & status information are periodically uploaded to ThingSpeak for cloud-based storage & visualization. An LED provides a temperature alert indication when a valid reading exceeds the configured demonstration threshold of 35°C.

AIM & OBJECTIVES: The aim is to develop an IoT-based patient monitoring prototype integrating physiological sensing, local visualization & cloud connectivity. The specific objectives are to:

- Interface the AD8232 ECG sensor, pulse sensor & DS18B20 temperature sensor with the ESP32.
- Acquire sensor signals & display available readings through a locally hosted web dashboard.
- Visualize the ECG signal using Chart.js.
- Upload selected sensor readings & status information to ThingSpeak periodically.
- Perform an LED alert when a valid temperature reading exceeds the configured 35°C demonstration threshold.
- Examine system behaviour through functional tests & actual observations.

SCOPE: The project focuses on sensor acquisition, local web-based monitoring, cloud data visualization & threshold-based LED indication. It demonstrates IoT integration for health monitoring purposes.

3. LITERATURE REVIEW & EXISTING SYSTEM

IoT-based patient monitoring systems integrate physiological sensors, embedded controllers & communication networks to collect & transmit health-related measurements. Existing approaches commonly use wearable sensors, microcontrollers & cloud platforms to support remote observation of physiological parameters. Their effectiveness depends on sensor quality, signal processing, network availability & the reliability of data transmission.

ECG monitoring systems use electrodes & signal-conditioning circuits to acquire electrical activity associated with the heart. The AD8232 is a single-lead ECG front-end designed for biopotential signal acquisition. Pulse sensors provide signals that can be processed to estimate heart rate, while digital temperature sensors such as the DS18B20 measure temperature & communicate the readings to a microcontroller. Integrating these sensors with an ESP32 enables local data acquisition & Wi-Fi connectivity within a compact prototype.

Research reviews identify network reliability, data quality, interoperability, privacy & security as important challenges in IoT-based healthcare monitoring. Andrade et al. (2024) examined vital-sign acquisition methods, communication protocols & monitoring architectures, highlighting opportunities to improve data transfer & system efficiency. Abdulmalek et al. (2022) also reviewed healthcare-monitoring technologies & identified security, privacy & quality-of-service concerns.

Table 1. Comparison of existing patient-monitoring approaches

APPROACH	MAIN FUNCTION	LIMITATION
Conventional monitoring	Local observation of physiological parameters	Limited remote accessibility
Single-sensor IoT system	Networked monitoring of one parameter	Limited physiological coverage
Cloud-based monitoring	Remote storage & visualization of measurements	Depends on network & cloud availability
Integrated IoT monitoring	Combines multiple sensors, local processing & cloud connectivity	Requires reliable sensor integration & data transmission

The technical specifications of the selected components support their use in the proposed prototype. The AD8232 provides a single-lead ECG front end with lead-off detection, while the DS18B20 offers digital temperature measurement through a 1-Wire interface. The ESP32 integrates Wi-Fi connectivity, enabling communication between the embedded system & network-based services.

TECHNICAL GAP: The project addresses the integration of ECG signal acquisition, pulse-rate estimation, temperature measurement, local web visualization, cloud-based data logging & threshold-based LED indication within a single educational prototype. Unlike systems focused only on local display or cloud reporting, the proposed design combines both monitoring interfaces.

4. PROPOSED SYSTEM & METHODOLOGY

The proposed system uses an ESP32 microcontroller to acquire physiological signals, process sensor readings & provide local & cloud-based monitoring. The AD8232 ECG sensor captures the electrical activity of the heart, the pulse sensor provides a signal for heart-rate estimation, & the DS18B20 measures temperature. The ESP32 hosts a web dashboard for local visualization & periodically uploads & readings & status information to ThingSpeak through Wi-Fi.

4.1 SYSTEM ARCHITECTURE

The system consists of four functional stages: sensing, processing, communication & visualization. The sensors provide inputs to the ESP32 through the assigned GPIO pins. The controller makes the acquired data available to the local dashboard & sends selected measurements to ThingSpeak. The temperature alert logic controls an LED when a valid temperature reading exceeds the configured threshold of 35°C.

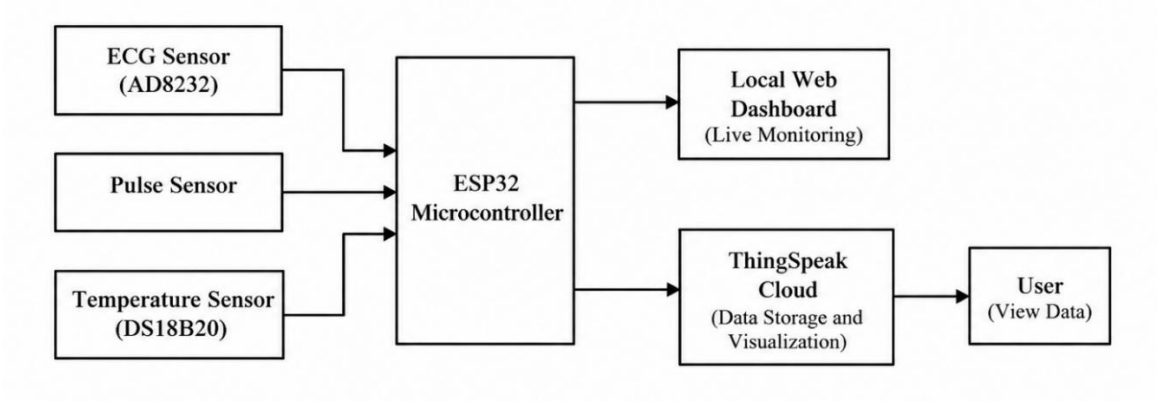


Figure 1. Architecture of the IoT-based patient monitoring system

4.2 METHODOLOGY

The system operates through the following sequence:

1. **Initialization**: The ESP32 initializes the sensors, LED output, Wi-Fi connection & local web server.
2. **Data acquisition**: The controller reads the ECG signal from the AD8232, the pulse signal from the pulse sensor & temperature measurements from the DS18B20.
3. **Data processing**: The acquired signals & readings are handled according to the requirements of each sensor. The pulse signal is used for heart-rate estimation, while the ECG signal is made available for waveform visualization.

4. **Temperature alert**: The ESP32 checks whether the temperature reading is valid & exceeds 35°C. If the condition is satisfied, the LED connected to GPIO 26 is activated; otherwise, it remains off.
5. **Local monitoring**: The ESP32 serves a web dashboard that displays available readings & visualizes the ECG waveform using Chart.js.
6. **Cloud communication**: Selected sensor readings & status information are periodically uploaded to ThingSpeak over Wi-Fi.
7. **Visualization**: The local dashboard presents sensor information & the ECG waveform, while ThingSpeak stores & plots periodically uploaded samples.

4.3 INPUT–PROCESSING–OUTPUT RELATIONSHIP

STAGE	DESCRIPTION
Input	ECG signal, pulse sensor signal & temperature readings
Processing	Sensor acquisition, pulse-rate estimation, temperature threshold comparison & data formatting
Communication	Wi-Fi, local HTTP-based dashboard access & ThingSpeak cloud updates
Output	Local sensor display, ECG waveform, cloud graphs & temperature alert LED

The 35°C threshold is a demonstration setting used to verify the programmed LED response; it is not a medical threshold. The cloud visualization represents periodic uploaded samples & does not constitute continuous ECG waveform monitoring.

5. HARDWARE & SOFTWARE REQUIREMENTS

The hardware components were selected to support physiological signal acquisition, embedded processing, wireless communication & temperature alert indication.

Table 2. Hardware components & their functions

COMPONENT	IMPORTANT SPECIFICATION	PURPOSE
ESP32 development board	Integrated Wi-Fi & GPIO interfaces	Sensor acquisition, processing & communication
AD8232 ECG sensor	Single-lead ECG signal acquisition	Acquisition of the ECG signal
Pulse sensor	Analog signal output	Heart-rate estimation
DS18B20 temperature sensor	Digital temperature sensor; 1-Wire interface	Temperature measurement
LED	Connected to GPIO 26	Temperature alert indication
USB power supply	Suitable for the ESP32 development board	Powering the prototype

The software environment supports microcontroller programming, sensor interfacing, dashboard development & cloud data visualization.

Table 3. Software environment & functions

SOFTWARE / PLATFORM	PURPOSE
Arduino IDE	Writing, compiling & uploading ESP32 firmware
ESP32 board support package	ESP32 compilation & programming support
OneWire library	Communication with the DS18B20 sensor
DallasTemperature library	Temperature acquisition & handling
HTML, CSS & JavaScript	Developing the local web dashboard
Chart.js	ECG waveform visualization
ThingSpeak	Cloud data storage & visualization
Web browser	Accessing the local dashboard & cloud interface

6. SYSTEM DESIGN & IMPLEMENTATION

The prototype was implemented by interfacing the sensors & LED with the ESP32 & developing firmware using the Arduino IDE. The firmware handles sensor acquisition, temperature alert logic, local web-server operation & periodic communication with ThingSpeak.

6.1 HARDWARE INTERFACING

The sensors & LED were connected to the GPIO pins of the ESP32 according to their signal requirements.

Table 4. ESP32 pin configuration

COMPONENT	ESP32 PIN	FUNCTION
AD8232 ECG output	GPIO 34	ECG signal input
AD8232 lead-off positive	GPIO 32	Positive lead-off detection
AD8232 lead-off negative	GPIO 33	Negative lead-off detection
Pulse sensor signal	GPIO 35	Pulse signal input
DS18B20 data	GPIO 4	Digital temperature communication
Temperature alert LED	GPIO 26	LED control output

The ESP32 acquires the ECG & pulse signals through its input pins & obtains temperature measurements through the OneWire interface. The DallasTemperature library supports temperature acquisition from the DS18B20. The LED is controlled through GPIO 26 according to the programmed temperature condition.

6.2 SENSOR DATA PROCESSING & ALERT LOGIC

The ECG signal acquired from GPIO 34 is made available for waveform visualization on the local dashboard. The pulse sensor signal from GPIO 35 is used for heart-rate estimation. Temperature measurements from the DS18B20 are checked for validity before the alert condition is evaluated.

When a valid temperature reading exceeds 35°C, the firmware activates the LED connected to GPIO 26. Otherwise, the LED remains off.

6.3 LOCAL WEB SERVER IMPLEMENTATION

The ESP32 hosts the monitoring dashboard through its web-server functionality. The dashboard is developed using HTML, CSS & JavaScript & accessed through the IP address assigned to the ESP32 by the Wi-Fi network.

The dashboard presents the available sensor readings & uses Chart.js to visualize the ECG waveform. JavaScript requests data from the ESP32's HTTP endpoints to update the displayed information. Access to the local dashboard requires the browser & ESP32 to be connected to the same Wi-Fi network.

6.4 CLOUD COMMUNICATION & HARDWARE ASSEMBLY

The ESP32 periodically transmits sensor readings & status information to ThingSpeak through its Wi-Fi connection. The cloud platform stores & visualizes these uploaded samples. Unlike the local dashboard, the cloud graph represents periodic data points rather than a continuous ECG waveform.

The hardware assembly consists of the ESP32 development board, AD8232 ECG sensor, pulse sensor, DS18B20 temperature sensor & LED connected according to the specified pin mapping. After completing the wiring & firmware configuration, the system is powered through a suitable USB supply & connected to Wi-Fi for monitoring.

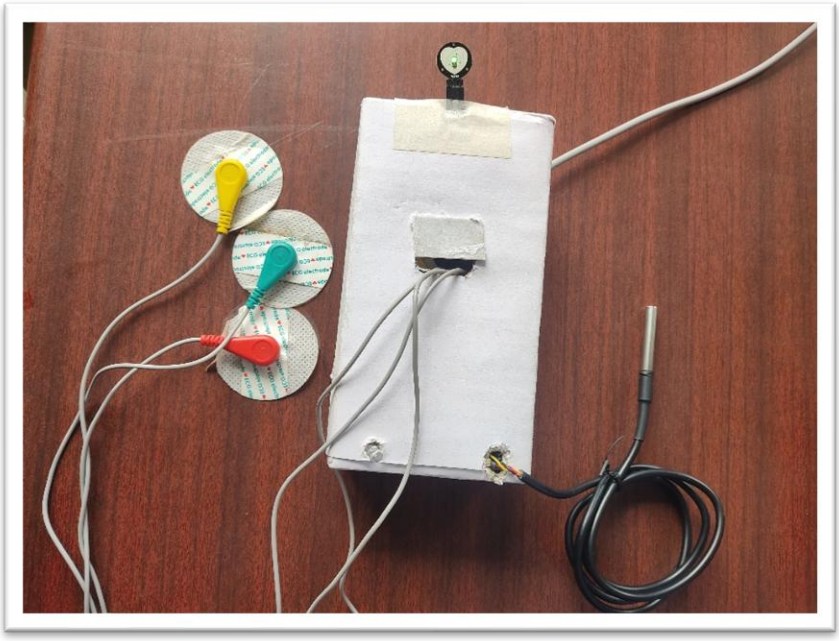


Figure 2. Hardware assembly of the IoT-based patient monitoring system

7. RESULTS & PERFORMANCE ANALYSIS

The performance of the IoT-based patient monitoring system is evaluated through sensor readings, local dashboard visualization, cloud data transmission & temperature alert behaviour.

7.1 LOCAL DASHBOARD RESULTS

The local web dashboard hosted by the ESP32 displays the available sensor readings & ECG waveform. Chart.js is used to visualize the ECG signal. The dashboard provides a local interface for observing sensor data without requiring a separate server.

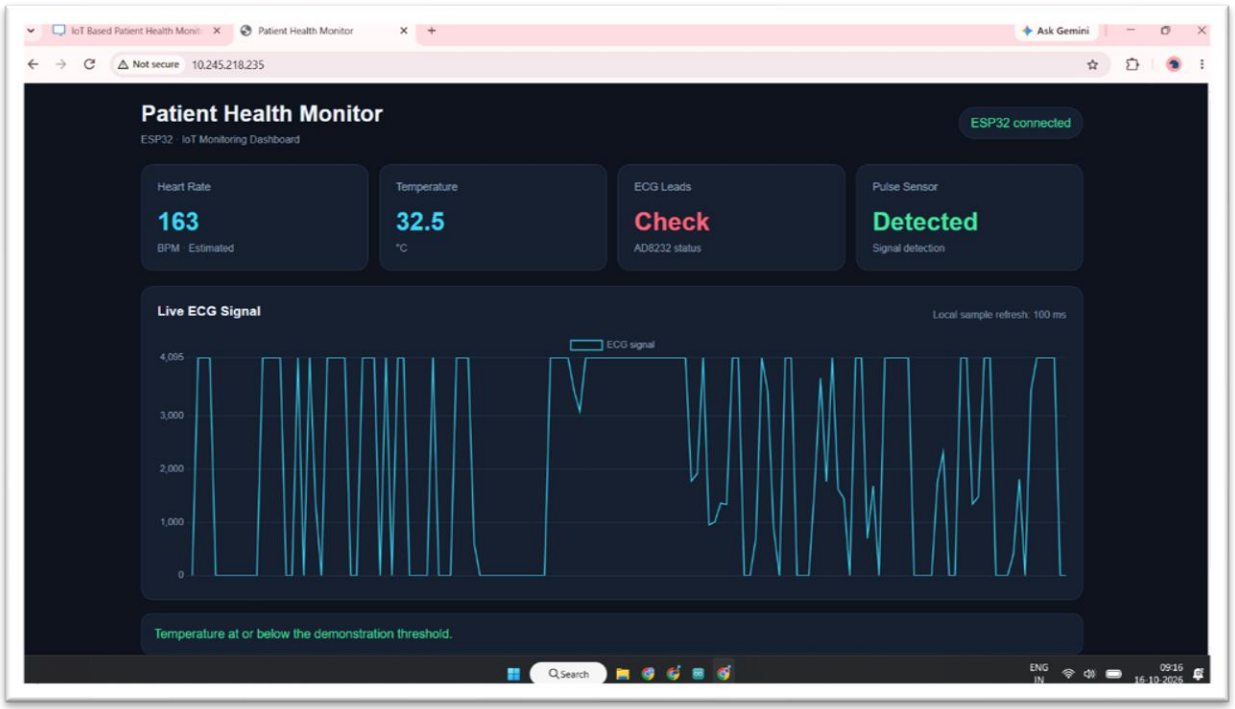


Figure 3. Local web dashboard

7.2 THINGSPEAK CLOUD VISUALIZATION

ThingSpeak receives selected sensor readings & status information periodically through the ESP32's Wi-Fi connection. The uploaded samples are stored & visualized as cloud graphs.

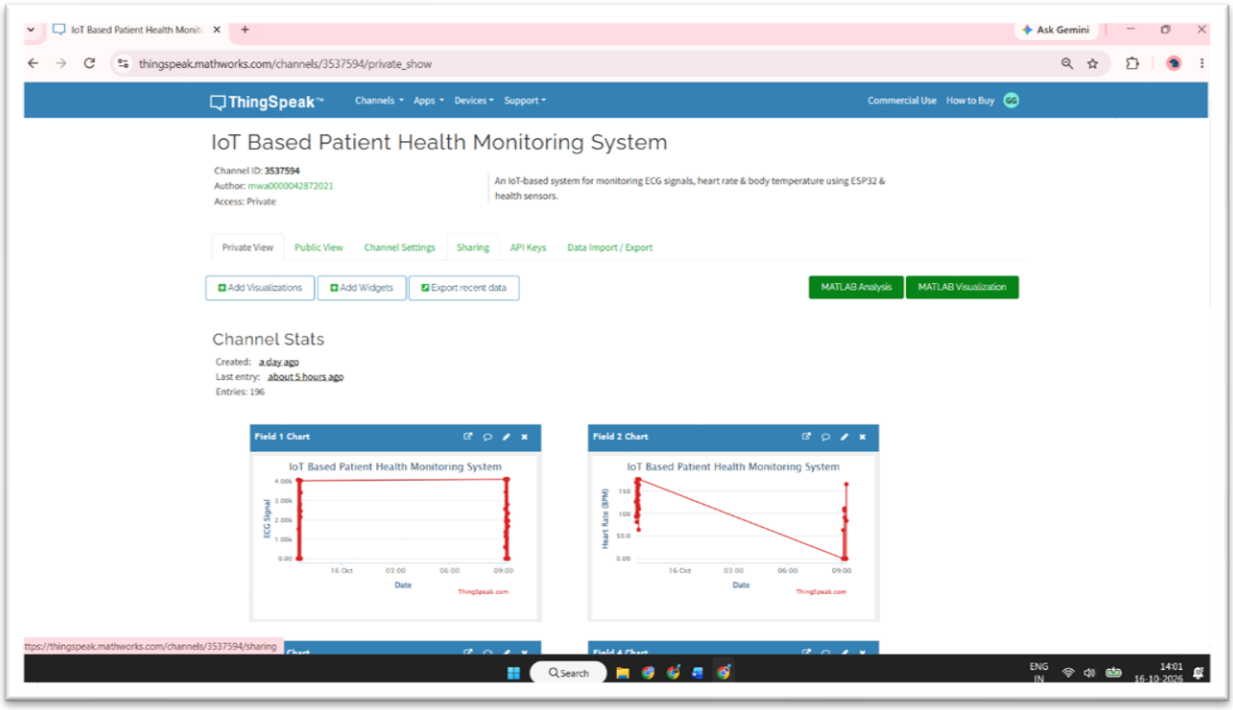


Figure 4. ThingSpeak cloud visualization

7.3 FUNCTIONAL TESTING

The following test cases can be used to evaluate the prototype. The observed responses & results are completed using actual test observations.

Table 5. Functional test cases

TEST CONDITION	RESPONSE
ECG sensor connected	ECG reading displayed on local dashboard
Pulse sensor connected	Heart-Rate reading displayed
Valid temperature reading available	Temperature displayed on dashboard
Valid temperature exceeds 35°C	LED connected to GPIO 26 activates
Wi-Fi & Cloud connectivity	Readings displayed on cloud dashboard

8. APPLICATIONS, ADVANTAGES & LIMITATIONS

APPLICATIONS

- Educational demonstration of IoT-based physiological monitoring.
- Local visualization of ECG signals & sensor readings.
- Cloud-based storage & visualization of periodically uploaded sensor data.
- Demonstration of threshold-based temperature alert systems.
- Experimental study of embedded systems, wireless communication & sensor integration.

ADVANTAGES

- Integrates multiple sensors using a single ESP32 microcontroller.
- Provides a locally hosted web dashboard without requiring a separate web server.
- Supports ECG waveform visualization using Chart.js.
- Enables cloud-based data storage & visualization through ThingSpeak.
- Provides a simple LED indication based on a configurable temperature threshold.

LIMITATIONS

- Local dashboard access depends on Wi-Fi connectivity & the ESP32's assigned IP address.
- Cloud updates depend on network availability & successful data transmission.
- Pulse-rate estimation & ECG visualization depend on signal quality & appropriate signal processing.
- The cloud interface displays periodic samples rather than a continuous ECG waveform.
- The 35°C threshold is intended for demonstration & is not a medical threshold.

9. CONCLUSION & FUTURE SCOPE

The IoT-based patient monitoring system integrates an ESP32 microcontroller with an AD8232 ECG sensor, a pulse sensor & a DS18B20 temperature sensor. The system is designed to provide local monitoring through a web dashboard, ECG waveform visualization using Chart.js, cloud-based data visualization through ThingSpeak & LED-based temperature alert indication. The LED alert is configured to activate when a valid temperature reading exceeds the demonstration threshold of 35°C.

The prototype demonstrates the integration of physiological sensing, embedded processing, Wi-Fi communication & web-based visualization. Its overall effectiveness must be established through the actual functional test results, including sensor readings, dashboard operation, cloud updates & LED response.

FUTURE SCOPE

- Improve ECG signal processing & pulse-rate estimation through appropriate filtering & validation.
- Add secure communication & authentication for cloud data transmission.
- Develop a mobile-friendly interface for convenient remote visualization.
- Implement data logging & analysis of historical sensor readings.
- Evaluate sensor performance & communication latency through systematic testing.
- Explore additional physiological sensors & validated alert mechanisms.

10. REFERENCES

- Abdulmalek, S., Nasir, A., Jabbar, W. A., Almuhaya, M. A. M., Bairagi, A. K., Khan, M. A.-M., & Kee, S.-H. (2022). IoT-based healthcare-monitoring system towards improving quality of life: A review. Healthcare, 10(10), Article 1993. https://doi.org/10.3390/healthcare10101993
- Analog Devices. (n.d.). AD8232: Single-lead heart rate monitors front end. https://www.analog.com/en/products/ad8232.html
- Analog Devices. (n.d.). DS18B20: Programmable resolution 1-Wire digital thermometer. https://www.analog.com/en/products/DS18B20.html
- Andrade, A., et al. (2024). IoT-based vital sign monitoring: A literature review. Smart Health, 32, Article 100462. https://doi.org/10.1016/j.smhl.2024.100462
- Chart.js. (n.d.). Chart.js documentation. https://www.chartjs.org/docs/latest/
- Espressif Systems. (n.d.). ESP32 series datasheet. https://www.espressif.com/en/support/documents/technical-documents
- MathWorks. (n.d.). ThingSpeak documentation. https://www.mathworks.com/help/thingspeak/